The `sendKeys()` method doesn't set the value of the field; rather, it simulates the user typing the text into the field. If the field contains an existing value, you'll need to use the `clear()` method before entering the new value.

The current value of a text field is stored in the `value` attribute. To retrieve an attribute value, you can use the `getAttribute()` method, as shown here:

```
String fromValue = driver.findElement(By.id("from")).getAttribute("value");
```

This approach will also work for any other form field that uses the `value` attribute, such as check boxes or the newer HTML5 input field types like `email` and `date`. The exception is `<textarea>` fields, which don't have a `value` attribute. You can retrieve the contents of a `<textarea>` field by using the `getText()` method.

Radio buttons and check boxes

The simplest way to select a radio button value is to find the radio button you want and to click on it. This can be a little tricky because the `name` attribute isn't unique and the `id` attribute isn't always defined or directly related to the value. You can do this by using a CSS selector that combines the name and value you want, like this:

```
driver.findElement(
       By.cssSelector("input[name='flightType'][value='return']")).click()
```

This approach will also work for check boxes, which behave in exactly the same way.

Drop-down lists

WebDriver provides a convenient helper class for dealing with drop-down lists. The `Select` class is used to wrap a web element representing a drop-down list to add drop-down-specific methods, such as `selectByVisibleText()`, `selectByValue()`, and `selectByIndex()`. On the booking page, you could set the travel class to "Business" using the following code:

```
WebElement travelClassElt = getDriver().findElement(By.id("travel-class"));
new Select(travelClassElt).selectByVisibleText("Business");
```

The `Select` class also provides a number of methods that you can use to learn about the current state of the drop-down list, including `getFirstSelectedOption()` and `getAllSelectedOptions()`.

8.2.4 Working with asynchronous pages and testing AJAX applications

Most modern web applications use AJAX in one way or another. AJAX-based JavaScript libraries allow developers to write applications with vastly improved usability and user experience. But the asynchronous nature of AJAX can present challenges when it comes to automated web testing.

In a conventional web application, when you click on a link or submit a form, an HTTP request is sent to the server and a new page is returned. In these cases, WebDriver will automatically wait for the new page to load before proceeding. But with an AJAX application, the web page will send queries to a server and update the page